



ISLAMIC UNIVERSITY OF TECHNOLOGY
A Subsidiary Organ of The Organization of Islamic Cooperation (OIC)



Report

Feather DB

Software Project Lab 1

SWE 4304

Third Semester, Session 2023-2024

Submitted to

Samia Nawsheen
Junior Lecturer

Submitted by

Md. Saif-Al Sarker
230042106
M Safwan Hasan Khan
230042117
Wasi Omar Syed
230042143

Date of Submission

14 December, 2025

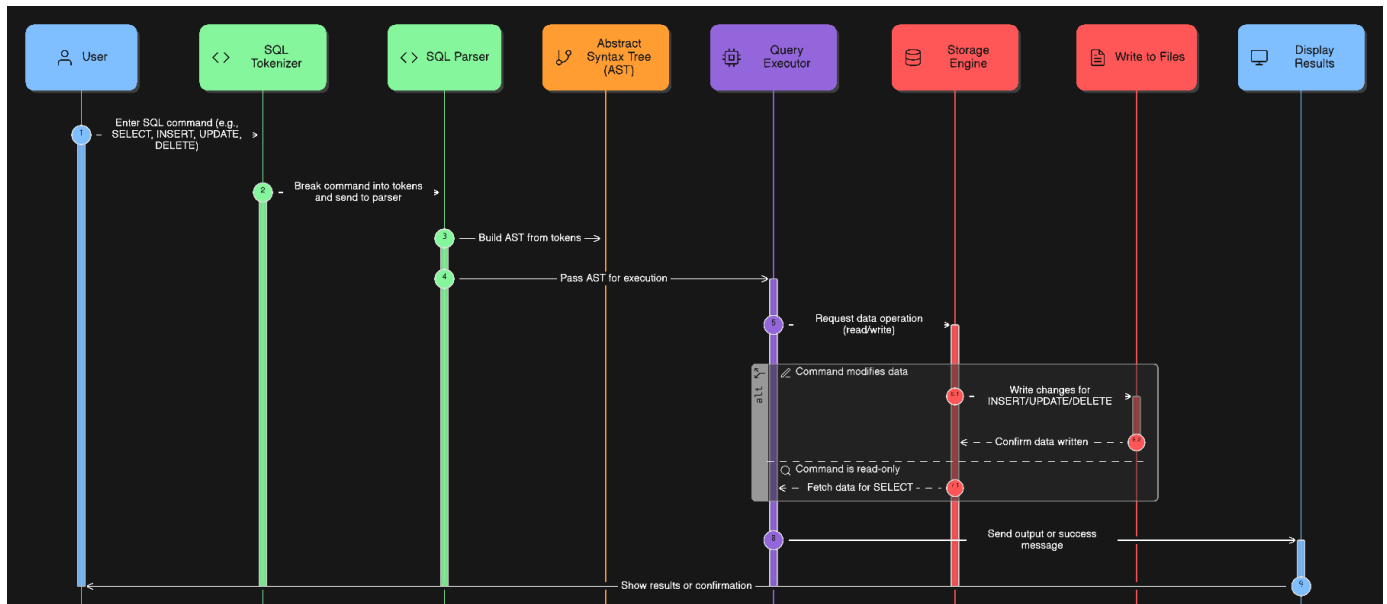
Project Overview

FeatherDB is a simple, file-based SQL database engine built from scratch using C++. It operates through a command-line interface (CLI), providing users with the ability to interact with the database using SQL-like commands. The project aims to implement core SQL functionalities such as CREATE, INSERT, SELECT, UPDATE, and DELETE while using custom-built parsers and file-based storage. The goal of the project is to provide hands-on experience in database system development, offering a practical learning tool for understanding SQL parsing, query execution, and file management in a database context.

Motivation Behind the Project

The motivation for developing FeatherDB is to gain practical experience in creating a database engine from the ground up, without relying on existing database management systems. This will provide valuable insight into the internals of SQL engines, including parsing commands, storing data, and executing queries. Additionally, the project allows the team to work with core database concepts such as file handling, data structures, and user interfaces. The project also aims to foster team collaboration, enhance problem-solving skills, and simulate a real-world software development environment.

Flow Chart



Key Features

Parser

The SQL parser is implemented using an Abstract Syntax Tree (AST), allowing efficient parsing and transformation of SQL commands into executable structures.

Table Creation

The system supports the definition of relational schemas through CREATE TABLE statements, allowing users to specify column names, data types, and related metadata.

Data Insertion

Data can be inserted into existing tables using INSERT INTO operations. Each insertion undergoes strict type validation to ensure consistency with the declared schema.

Data Retrieval

The system enables data retrieval via the SELECT command, providing the ability to access stored records in a structured and efficient manner.

Data Modification

Modification of existing records is supported through UPDATE and DELETE operations, allowing targeted alterations or removal of specific tuples.

WHERE Clause Support

Conditional filtering is implemented through the WHERE clause, enabling selective retrieval or modification of records based on user-defined logical expressions.

Nested Query Support

The parser and execution engine support nested and subquery constructs, allowing complex queries to be composed and evaluated within broader SQL expressions.

Role-Based Access Control (RBAC)

The system includes Role-Based Access Control (RBAC), allowing administrators to define user roles and permissions, ensuring that only authorized users can perform certain actions (such as inserting, updating, or deleting records). This enhances security by restricting access based on user roles.

Tools & Technology

The following technologies and tools will be utilized for the development of the FeatherDB project:

- **Programming Language:** C++. Following feedback received during the project proposal presentation, the team has decided to implement the database engine in C++. This choice provides greater control over memory management and system resources, which is crucial for building a lightweight, efficient database system from scratch.
- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code). VS Code will be used as the primary editor for writing, debugging, and managing the C++ source code due to its powerful features, extensive plugin ecosystem, and cross-platform compatibility.
- **Version Control System:** Git. Git will be employed to manage the source code repository, enabling seamless collaboration among team members, tracking changes, and maintaining a history of the project's evolution.
- **Code Hosting Platform:** GitHub. The project repository will be hosted on GitHub, facilitating collaborative development, code reviews, issue tracking, and providing a central location for all project assets.

Proposed Timeline

The project spans fourteen weeks in total. The work plan is structured to ensure steady progress from foundational design to full implementation, testing, and presentation. Each phase contributes directly to the core objective of constructing a minimalist, C++-based SQL-like database engine with custom parsing, storage, and execution components.

FeatherDB		Software Project Lab 1											
Team 4													
SL	TASK TITLE	WEEK 1-2	WEEK 3-4	WEEK 5	WEEK 6	WEEK 7	WEEK 8	WEEK 9	WEEK 10	WEEK 11	WEEK 12	WEEK 13	WEEK 14
1	Initial Planning & Team Formation												
2	Proposal Development and Initial Setup												
3	Core Fundamentals and Foundational Components												
4	Storage Engine and Parser Construction												
5	Command-Line Interface (CLI) Development												
6	Extended Features and Role-Based Access Control												
7	Testing, Documentation, and Final Presentation												

Weeks 1–2: Planning and Team Formation

The first two weeks were dedicated to establishing the project’s direction through brainstorming sessions, reviewing expectations, discussing feasibility, and forming the project team. During this phase, the team finalized the goal of building a lightweight database engine with minimal external library usage and clarified individual responsibilities.

Weeks 3–4: Proposal Development and Initial Setup

These weeks focused on preparing the project proposal and presenting it to the instructors. The feedback received during this stage guided refinements to the project scope, vocabulary used in parser design, and the plan to transition from Python to C++.

In parallel, the team began setting up the initial project structure in C++, organizing modules, selecting coding conventions, and preparing the environment for test-driven development.

Weeks 5–6: Core Fundamentals and Foundational Components

This stage is dedicated to laying the technical foundation of the system. The team will implement the core utilities required for a minimal database engine, including:

1. Custom string handling utilities (if required by the final design),
2. Lexing / tokenization logic,
3. Preliminary interfaces for parser modules,
4. Initial abstractions following object-oriented principles (e.g., Token, Node, Storage Manager interfaces).

The goal of this phase is to ensure that subsequent layers (storage and parsing) have a stable, well-defined base.

Weeks 7–8: Storage Engine and Parser Construction

The storage subsystem will be developed during this period, focusing on file-based persistence, record formatting, and basic table operations.

Alongside the storage layer, the team will work on the parser—constructing the Abstract Syntax Tree (AST) with at least three levels of nesting, refining syntactic rules, and establishing the link between parsed queries and upcoming execution logic.

A *mid-project progress presentation* is planned between Weeks 8–9 to evaluate alignment with expectations and receive additional feedback.

Weeks 9–10: Command-Line Interface (CLI) Development

These weeks are devoted to building the interactive command-line interface that users will interact with.

Tasks include:

- Designing input handling and communication with the parser,
- Providing meaningful error messages,
- Integrating the parser with the storage and execution pipeline,
- Ensuring smooth round-trip execution of simple queries.

This phase turns the internal modules into a usable tool.

Weeks 11–12: Extended Features and Role-Based Access Control

After the MVP becomes functional, focus shifts toward the additional features recommended during the proposal feedback. These include:

- designing a role-based access control (RBAC) system, such as admin/user roles with permission boundaries;
- supporting more complex SQL-like operations (e.g., nested queries aligned with the three-level AST requirement);
- strengthening internal modularity and improving parser vocabulary and error handling.

These enhancements expand both the realism and academic value of the project.

Weeks 13–14: Testing, Documentation, and Final Presentation

The final two weeks concentrate on rigorous testing following the test-driven development approach. Unit tests, integration tests, and behavior tests will verify correctness across all components—lexing, parsing, storage, CLI, and RBAC.

Simultaneously, the team will prepare project documentation, including design explanations, module descriptions, usage guidelines, and analysis of limitations.

The work concludes with the final project presentation, demonstrating the database engine's features, architecture, and evolution from proposal to implementation.

Action Based on the Feedback

During the proposal presentation, our team received valuable feedback from the instructors. This report outlines how that feedback has shaped the updated direction of our project and the concrete steps we are taking to improve comprehensiveness, technical depth, and alignment with the course outcomes.

Response to Presentation Feedback

1 Transition from Python to C++

The instructors recommended moving from Python to C++ to deepen our low-level understanding and gain more control over memory, data structures, and runtime behavior. Following this feedback, the entire project has been shifted to C++. This transition also aligns naturally with our initial goal of minimizing library usage since C++ allows fine-grained control over core components.

As a result, our implementation strategy now includes writing essential utilities from scratch, such as:

- a custom string class to replace `<string>` where feasible,
- manual tokenization routines,
- a hand-built parser without relying on parser generators or advanced libraries.

This shift ensures that each component of the database engine emerges from our own logic and design rather than from existing abstractions.

2 Increasing Comprehensiveness

The instructors emphasized that the proposal felt closer to a lab task than a full project. To address this, we extended the scope, depth, and academic rigor of our design.

We are integrating object-oriented concepts (OOC) throughout the architecture, using abstractions such as token streams, AST (Abstract Syntax Tree) nodes, executors, storage engines, and error handlers. The project will also follow test-driven development (TDD) to ensure reliability, traceability, and systematic progress.

On the technical side, we are expanding the feature set to reflect an actual miniature database system rather than a collection of simple functions. These enhancements include:

- constructing multi-level parse trees (minimum three layers of nesting),
- supporting nested SQL-like queries,
- designing a modular execution engine,
- implementing a file-based storage layer for persistence,
- introducing a basic query planner and evaluator.

Each of these components improves the comprehensiveness and realism of the system.

3 Vocabulary and Structural Improvements in Parser Design

Both instructors noted the need to refine our parser vocabulary and the hierarchy of parsing layers. In response, we are restructuring the parser into clear, well-defined stages: lexical analysis, syntactic analysis, and semantic resolution. The Abstract Syntax Tree will explicitly enforce at least three nested layers to support more complex queries and demonstrate a deeper understanding of language processing.

4 Role-Based Access Control (RBAC)

The instructors suggested enhancing the project by adding a role-based access system. After completing the MVP, we will implement user roles such as admin and regular user, each with different permissions. For example, administrative roles may create or modify tables, while regular users may only run read-only queries. This expands the project's practical relevance and technical complexity.

5 Updated Project Scope

Based on all feedback, the updated scope now includes:

- C++-based implementation with minimal library dependency,
- custom-built string handlers, tokenizers, and parsers,
- 3-layer minimum AST structure,
- support for nested SQL-like queries,
- modular OOC-based architecture,
- TDD throughout development,
- persistence layer for storing table data,
- optional RBAC module after the MVP is complete.

This expanded scope ensures that the project is no longer “small” or “lab-like,” but rather a meaningful exploration of database internals.